Flappy Bird Based on Q-Learning

Youtube: https://youtu.be/m6utf7LwzXk

Github: moskan/Q-learning-flappy-bird

1 Introduction

This project implements a reinforcement learning agent using the Q-learning algorithm to play a Flappy Bird-like game. The agent learns from scratch how to navigate through pipes without collisions by interacting with the environment. We utilized PyGame to construct the user interface (UI) and employed Q-learning to train the AI for 10,000 games, thereby enhancing the AI's game-playing proficiency. Pygame is a library for game development that provides functions such as graphics, sound, and input processing. We use this library to implement game operation, dynamic updates, event detection, and UI rendering..

2 Game Design

2.1 Rules of the Game

In the traditional flappy bird game, players tap the touch screen, and the bird will fly upwards. Continuous taps will send it higher and higher. Releasing the finger will cause it to descend rapidly. The frequency and timing of taps are crucial for controlling the bird's flight. Therefore, players need to control the bird to fly forward continuously, while also avoiding uneven pipes along the way. The bird has acceleration when flying, making each tap cause it to fly in an arc. The key to

AI training is to issue flight commands at the right time.

2.2 Class Design of the Game

The project employs an object-oriented design with two core classes: the FlappyBirdEnv(environment) and the QLearningAgent.

FlappyBirdEnv Class: This class encapsulates the entire game world, its physics, and its rules. Its key responsibilities include:State Management: It discretizes the continuous game screen into a manageable state representation for the Q-learning agent. The state is a tuple of (horizontal_distance_to_next_pipe, vertical_distance_to_pipe_gap, bird_velocity_state).

Game Logic (step, _game_step): It handles the core game loop: updating the bird's position based on gravity and actions, moving and spawning pipes, and performing collision detection.

Reward Shaping: It implements the reward function, providing +10for passing a pipe, +0.1for surviving a timestep, and a large penalty (-1000) for collisions.

Rendering (_render, _draw_*methods): When rendering is enabled, it manages the Pygame display, drawing all graphical elements and UI screens.

QLearningAgent Class: This class is the "brain" of the AI player. Its design focuses on the Q-learning algorithm:

Q-table Storage: It uses a Python dictionary to store Q-values, efficiently handling a sparse state-action space. The key is a (state, action)tuple.

Decision Making (choose_action): It implements an epsilon-greedy policy. With

probability epsilon, it explores (chooses a random action); otherwise, it exploits by choosing the action with the highest Q-value for the current state.

Learning (learn): It implements the core Q-learning update rule: $Q(s,a) = Q(s,a) + \alpha * (r + \gamma * \max_{a'} Q(s',a') - Q(s,a))$.

Persistence (save, load): It can save the learned Q-table to a file and load it later using the picklemodule, allowing training and demonstration to be separate phases.

This clean separation adheres to the standard Reinforcement Learning paradigm where the Agent interacts with an Environment, making the code modular, maintainable, and easy to understand.

2.3 UI Design

The overall user interface follows a simple style. The UI is built with Pygame and features three distinct screens for an engaging user experience: a Start Screen, a Game Play screen, and a Game Over screen.

Start Screen: This is the first interface users see. It features a pulsating title "Q-FLAPPY BIRD", a brief description of the project as a "Reinforcement Learning Demo", and clear instructions. A list of controls s provided. To guide the user, a blinking "PRESS SPACE TO START" prompt is displayed. The screen also shows the all-time HIGH SCORE, connecting the current session to past performances. A subtle animated background with moving clouds enhances visual appeal without being distracting.

Game Play Screen (HUD): During active play, the UI transforms into a Heads-Up Display (HUD). The current Scoreis prominently displayed in the top-left corner, alongside the Highscore. Crucially, for educational and debugging purposes, the HUD also shows real-time AI metrics: the Rewardfor the last action (colored green for positive, red for negative) and the last Actiontaken (JUMPor FALL). This provides a transparent view into the AI's decision-making process. The game world itself is rendered with visual details: a gradient sky, an animated bird with flapping wings, and pipes with a color gradient for depth.

Game Over Screen: Upon collision, the game transitions to this screen. A prominent "GAME OVER" message is shown. The player's final SCOREand the updated HIGH SCOREare displayed. Additional statistics, like the number of pipes passed and the last reward, are shown to summarize the episode. A blinking "PRESS SPACE TO RESTART" prompt invites the user to try again, creating a seamless loop. The use of semi-transparent overlays for the Start and Game Over screens focuses the user's attention on the UI elements while keeping the game world visible in the background.

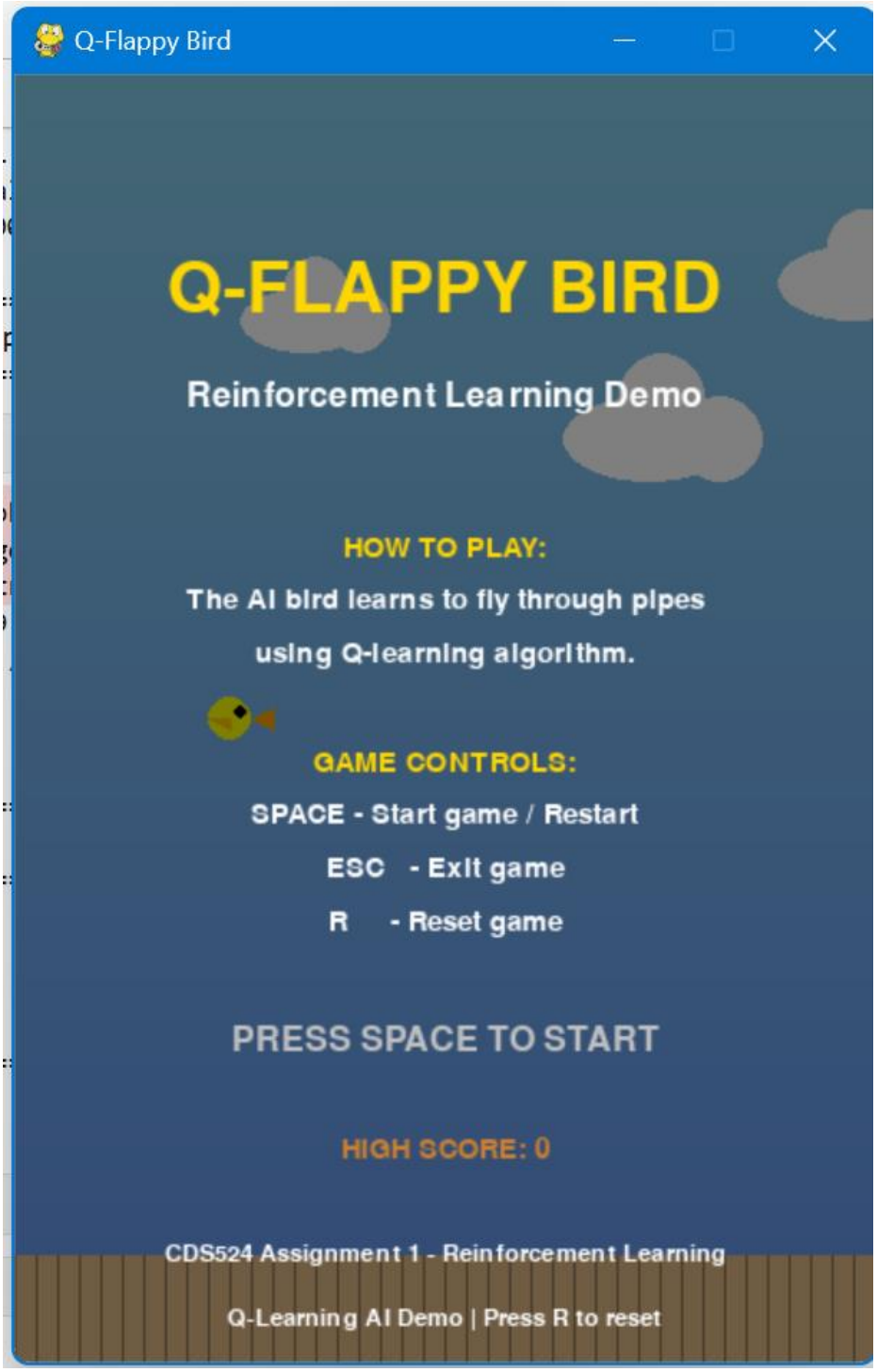

Fig 2.1 Flappy bird UI-1

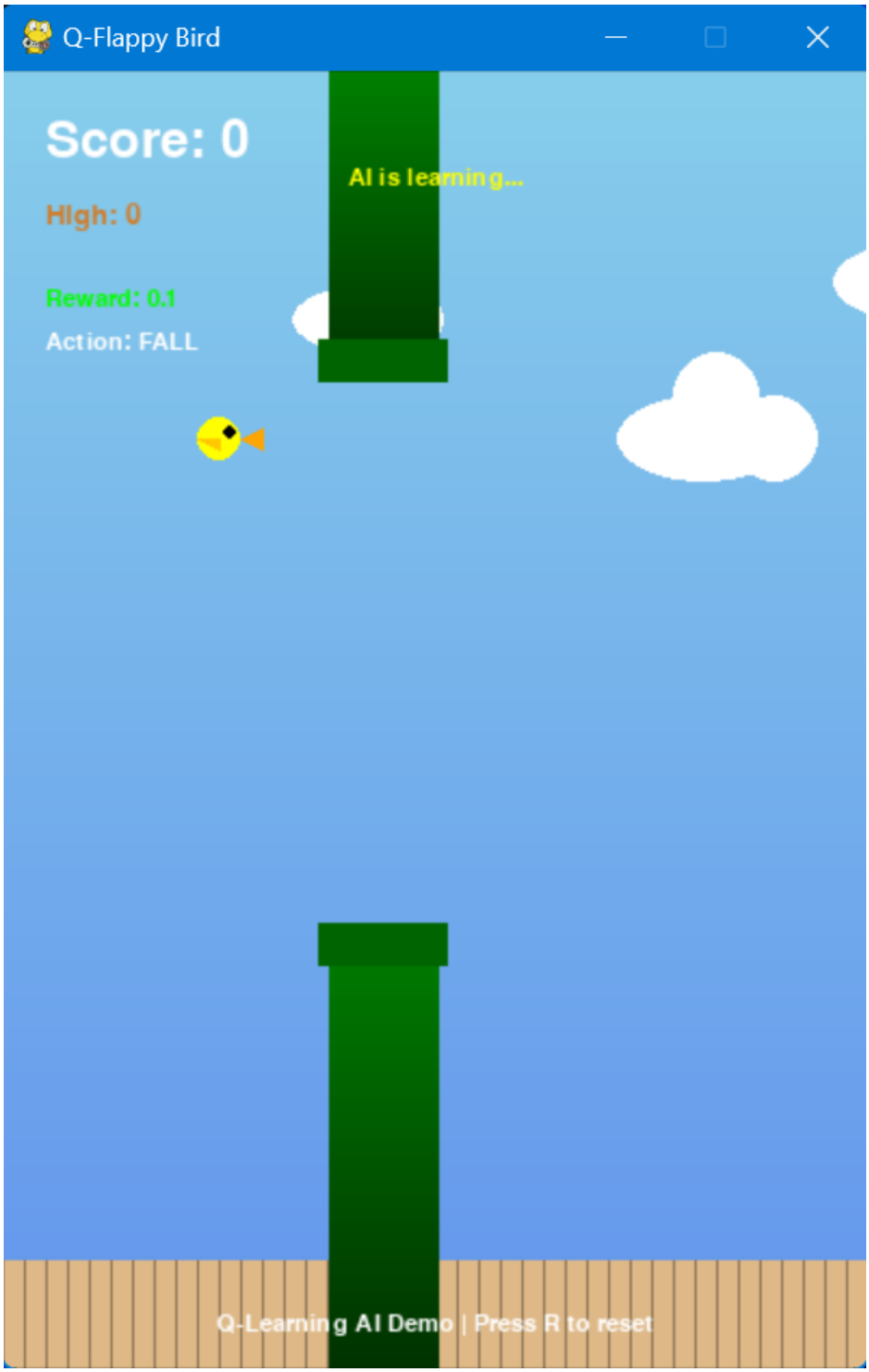

Fig 2.2 Flying

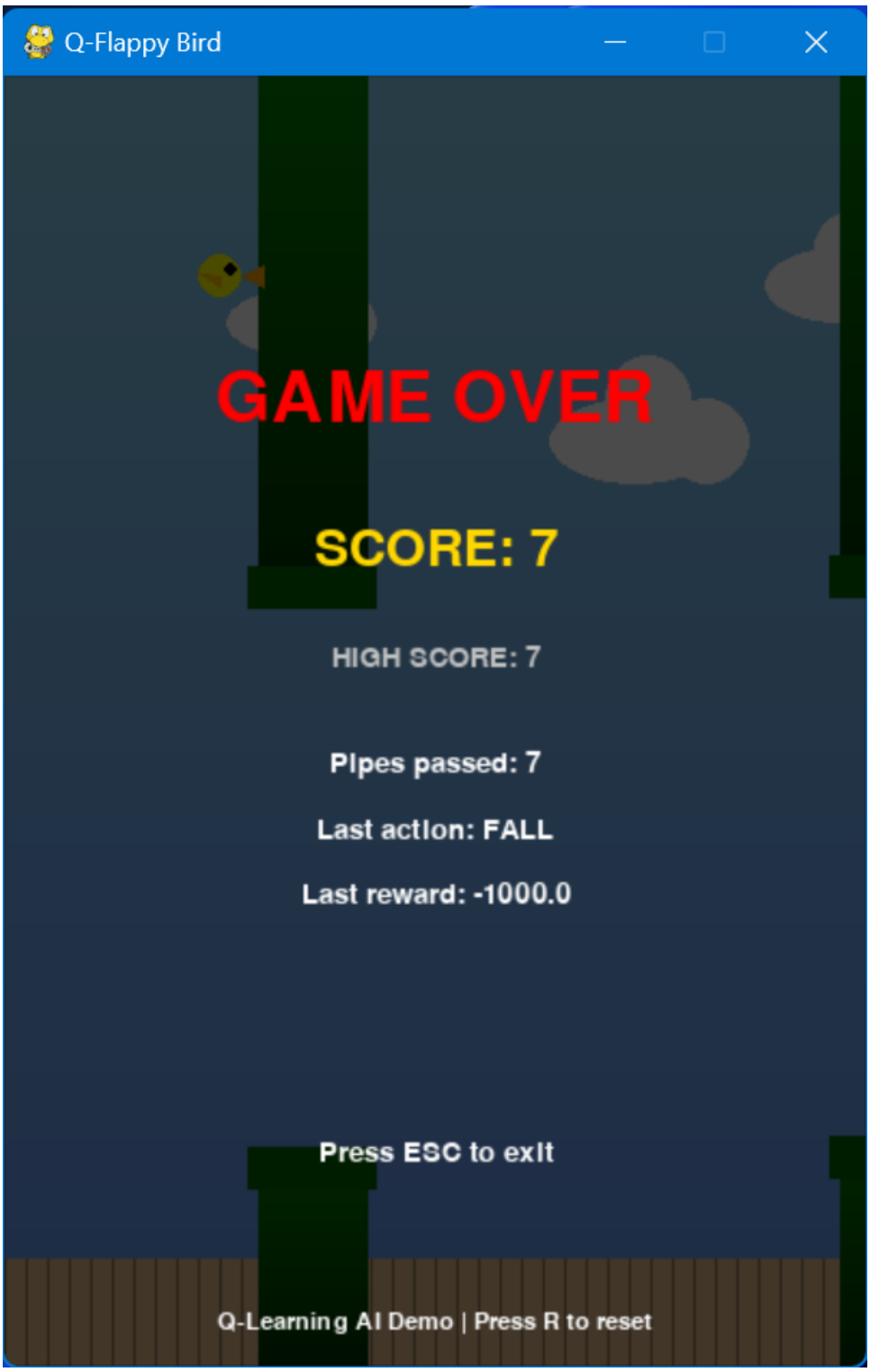

Fig 2.3 Game Over

3 Implement of Q-learning Algorithm

The Q-learning algorithm is implemented in the QLearningAgentclass. The implementation follows the standard temporal-difference learning approach for model-free Reinforcement Learning.

State and Action Space: The state space Sis discrete and finite, derived from the _get_state()method of the environment. An action space Ais simple: {0: Do nothing, 1: Jump}. The core of the agent is a Q-table, implemented as a Python dictionary q_table. The key is a tuple (s, a), and the value is the learned Q-value, representing the expected future reward for taking action ain state s.

Core Learning Loop: The learning occurs in the learn(self, state, action, reward, next_state, done)method.

1. It retrieves the current Q-value: current_q = self._get_q_value(state, action).

2. It calculates the target value. If the episode is done (done==True), the target is just the immediate reward. Otherwise, it is the discounted future reward: target = reward + self.gamma * max_future_q, where max_future_qis the maximum Q-value for the next state next_state.

3. It updates the Q-value using the Q-learning update rule: new_q = current_q + self.lr * (target - current_q). Here, lr(α) is the learning rate, controlling how much new information overrides the old.

4. The new value is stored back in the q_table.

Exploration vs. Exploitation: This is managed by the epsilon(ε) parameter in the choose_actionmethod. The agent starts with a high epsilon(e.g., 1.0), meaning it primarily explores by taking random actions. After each episode, epsilonis decayed by a factor (epsilon_decay, e.g., 0.995) until it reaches a minimum floor (epsilon_min, e.g., 0.01). This decay schedule ensures the agent explores the environment widely initially and gradually shifts to exploiting the learned policy. The discount factor gamma(γ), set to 0.99, makes the agent favor long-term reward over immediate gain.

3 Challenges and Solutions

Challenge 1: Pygame Font Initialization Bug on Windows

Problem: During UI development, calling pygame.font.SysFont(None, size)caused a crash on Windows. The error was TypeError: expected str, bytes or os.PathLike object, not int, originating from Pygame's internal system font enumeration code.

Solution: We abandoned the use of SysFontfor dynamic font creation. Instead, all required fonts are initialized once in the _init_fonts()method using pygame.font.Font(None, size). The Fontconstructor with Noneuses Pygame's default font and does not trigger the buggy system font scan. For visual effects like a pulsating title, we pre-create two font objects of different sizes and switch between them, rather than dynamically creating a font with a calculated size.

Challenge 2: Integrating Interactive UI with Agent Control Logic

Problem: The initial test script had a monolithic loop. When the user pressed SPACE on the start screen, the game would start, but the main loop would continue running its fixed number of steps and then exit, not allowing proper gameplay or restart.

Solution: The game state machine was made explicit, and the main loop in test.pywas refactored into three distinct phases controlled by env.game_state("start", "playing", "game_over"). Each phase has its own event handling and rendering logic. The "start" and "game_over" phases wait for user input (SPACE) in a tight loop, while the "playing" phase delegates action selection to the trained QLearningAgent. This creates a clear, responsive, and user-friendly control flow.

Challenge 3: Balancing Simulation Speed (Training) vs. Rendering (Demo)

Problem: Training requires thousands of game episodes to be run quickly, but the final demo needs smooth, real-time rendering for the video presentation. Running Pygame rendering during training would slow it down immensely.

Solution: The FlappyBirdEnvconstructor accepts a render_modeparameter. In train.py, it is set to None, disabling all graphical operations and allowing the training to run at maximum speed. In test.py, it is set to "human", enabling the full Pygame rendering pipeline and UI. This parameterized design cleanly separates performance-critical training from the visual demonstration.

4 Conclusion

This project successfully designed and implemented a "Q-Flappy Bird" game that demonstrates the core principles of Reinforcement Learning, specifically the Q-learning algorithm. A custom game environment was built from scratch with a well-defined state and action space. A tabular Q-learning agent was implemented to learn an optimal policy for navigating through pipes. The project includes a fully-featured graphical user interface with start, gameplay, and game over screens, satisfying all interaction requirements.

The key achievement is the integration of a classic RL algorithm with a playable game and an engaging UI. The agent successfully learns, as evidenced by its increasing score over training episodes, and the saved policy allows for immediate demonstration. Challenges like platform-specific library bugs and state

management were diagnosed and resolved, resulting in a robust final product. This project serves as a comprehensive proof-of-concept for applying value-based reinforcement learning to simple game environments and provides a solid foundation for extensions, such as implementing Deep Q-Networks (DQN) for handling more complex state spaces.